Security Audit Report — AgiCardsRegistry

Project: AgiCards
Contract: `AgiCardsRegistry.sol`
Network: 0G Mainnet · `0xc757698204543af249e328764e89530464de668e`
Audit Date: 2026-05-16
nSLOC: 245
Auditor: Pashov AI Audit

Table of Contents

---

1. Executive Summary

AgiCardsRegistry is a single-contract ETH escrow and proof registry that lets users fund a wallet, register AI agents, set spending policies, and authorize card-spend requests. All state transitions are anchored by 0G Merkle proof roots.

The pre-audit x-ray analysis classified the codebase as **EXPOSED**: a single developer wrote all 245 nSLOC in one commit with no Solidity test coverage, no peer review, and two confirmed on-chain gaps in core fund-flow paths. The full solidity audit confirmed those gaps and surfaced additional findings.

9 findings were identified. 8 have been fixed. 1 remains open (low-impact DoS).

Severity	Total	Fixed	Open
High	1	1	0
Medium	5	5	0
Low	3	2	1

---

2. Protocol Overview

```
User
  └─ depositFunds(receiptRoot)        ← ETH credited to
depositedBalance[msg.sender]
  └─ withdrawFunds(amount)            ← available balance
withdrawn; CEI + nonReentrant
  └─ registerAgent(agentId, root)     ← agent identity anchored to
0G root
  └─ createPolicy(policyId, ...)      ← spending limits per agent
  └─ requestCard(requestId, ...)      ← reserves funds; checks
daily quota
       └─ approveCardRequest()        → Approved
            └─ logWeb3Spend()         → Completed; ETH routed to
treasury
            └─ logStripeAuthorization() → Completed; ETH routed to
treasury
       └─ rejectCardRequest()         → Rejected; quota + reserved
funds restored
       └─ releaseReservedFunds()      → Cancelled; quota +
reserved funds restored
```

Trust model: No protocol admin, no timelock, no multi-sig. Every action is gated solely by resource ownership (`onlyAgentOwner`, `requestOwner`). The same EOA creates and approves its own requests — the approval step is a lifecycle marker, not an independent authorization.

---

3. Scope

File	nSLOC	Commit
`contracts/AgiCardsRegistry.sol`	245	`a3409bb` → `bbaa845`

---

4. Findings Summary

#	Title	Severity	Status
F-1	Spent ETH permanently locked — no egress in `_consumeReserved`	High	Fixed
F-2	Daily spending limit never enforced on-chain	Medium	Fixed
F-3	`withdrawFunds` missing reentrancy guard	Medium	Fixed
F-4		Medium	Fixed

#	Title	Severity	Status
	`logWeb3Spend` accepts Stripe-mode requests		
F-5	Agent pause not checked in log functions	Medium	Fixed
F-6	Zero-spend allowed in log functions	Medium	Fixed
F-7	Cancelled/rejected requests permanently consume daily quota	Low	Fixed
F-8	`linkStripeCard` / `createWeb3Card` allow repeated calls	Low	Fixed
F-9	`requestId` slot poisoning via front-running	Low	Open

5. Detailed Findings

F-1 · High · Spent ETH Permanently Locked

Location: `AgiCardsRegistry._consumeReserved`
Confidence: 90
Status: Fixed in `bbaa845`

Description

`_consumeReserved` debits `depositedBalance[owner]` by `spentAmount` but sends no ETH out of the contract. All value logged via `logWeb3Spend` or `logStripeAuthorization` accumulated permanently in the contract with no admin sweep, fee collector, or user recovery path.

Fix Applied

```
+   address public immutable treasury;
+
+   constructor(address _treasury) {
+       require(_treasury != address(0), "zero treasury");
+       treasury = _treasury;
+   }

    function _consumeReserved(bytes32 requestId, uint256
        spentAmount) internal {
        ...
        depositedBalance[cardRequest.owner] -= spentAmount;
```

```
+       if (spentAmount > 0) {
+           (bool sent,) = treasury.call{value: spentAmount}("");
+           require(sent, "treasury transfer failed");
+       }
    }
```

F-2 · Medium · Daily Spending Limit Never Enforced

Location: `AgiCardsRegistry.requestCard`
Status: Fixed in `fe25d39`

Description

`Policy.dailyLimit` was written in `createPolicy` and validated (`dailyLimit >= maxPerRequest`) but never read again. `requestCard` only checked `amount <= maxPerRequest`. Any number of requests within `maxPerRequest` per call could be created in a single day, trivially exceeding the configured daily cap.

Fix Applied

```
+   mapping(bytes32 => mapping(uint256 => uint256)) public
        dailySpent;

    function requestCard(...) external onlyAgentOwner(agentId) {
        ...
+       uint256 today = block.timestamp / 1 days;
+       require(dailySpent[policyId][today] + amount <=
        policy.dailyLimit, "daily limit exceeded");
+       dailySpent[policyId][today] += amount;
        reservedBalance[msg.sender] += amount;
    }
```

F-3 · Medium · `withdrawFunds` Missing Reentrancy Guard

Location: `AgiCardsRegistry.withdrawFunds`
Status: Fixed in `fe25d39`

Description

`withdrawFunds` used a low-level `.call{value: amount}("")` with no reentrancy lock. Although CEI was followed for the direct path, there was no protection against cross-function re-entry through other state-mutating functions.

Fix Applied

```
-   function withdrawFunds(uint256 amount, bytes32 receiptRoot)
        external {
+   function withdrawFunds(uint256 amount, bytes32 receiptRoot)
        external nonReentrant {
```

F-4 · Medium · `logWeb3Spend` Accepts Stripe-Mode Requests

Location: `AgiCardsRegistry.logWeb3Spend`
Status: Fixed in `fe25d39`

Description

`logWeb3Spend` had no `stripeMode` check. A Stripe-mode request (intended to go through `logStripeAuthorization` and `linkStripeCard`) could be finalized via the Web3 path, bypassing the Stripe card linkage step and producing a divergent audit trail.

Fix Applied

```
    function logWeb3Spend(...) external requestOwner(requestId) {
        ...
+       require(!cardRequest.stripeMode, "use
        logStripeAuthorization");
```

F-5 · Medium · Agent Pause Not Checked in Log Functions

Location: `AgiCardsRegistry.logWeb3Spend`, `AgiCardsRegistry.logStripeAuthorization`
Status: Fixed in `fe25d39`

Description

`pauseAgent` set `agents[agentId].paused = true`, but only `requestCard` checked this flag. Approved in-flight requests could still be finalized by calling `logWeb3Spend` or `logStripeAuthorization` even after the agent was paused — defeating the intended emergency stop.

Fix Applied

```
    function logWeb3Spend(...) {
        ...
+       require(!agents[cardRequest.agentId].paused, "agent
        paused");

    function logStripeAuthorization(...) {
        ...
+       require(!agents[cardRequest.agentId].paused, "agent
        paused");
```

F-6 · Medium · Zero-Spend Allowed in Log Functions

Location: `AgiCardsRegistry.logWeb3Spend`, `AgiCardsRegistry.logStripeAuthorization`
Status: Fixed in `fe25d39`

Description

Both log functions accepted `amount = 0`. `_consumeReserved(requestId, 0)` would mark the request `Completed`, clear all reserved funds back to the user, and debit `depositedBalance` by zero — effectively marking a request complete with no actual spend recorded.

Fix Applied

```
+   require(amount > 0, "invalid spend amount");
```

F-7 · Low · Cancelled/Rejected Requests Permanently Consume Daily Quota

Location: `AgiCardsRegistry._releaseReserved`
Status: Fixed in `bbaa845`

Description

`dailySpent[policyId][today]` was incremented in `requestCard` but never decremented when a request was cancelled or rejected. Creating and cancelling requests permanently burned the daily quota even though no ETH was spent.

Fix Applied

```
    function _releaseReserved(bytes32 requestId) internal {
        ...
        if (amount > 0) {
            cardRequest.reservedAmount = 0;
            reservedBalance[cardRequest.owner] -= amount;
+           dailySpent[cardRequest.policyId]
         [cardRequest.createdAt / 1 days] -= amount;
            emit FundsReleased(...);
        }
    }
```

F-8 · Low · `linkStripeCard` / `createWeb3Card` Allow Repeated Calls

Location: `AgiCardsRegistry.linkStripeCard`, `AgiCardsRegistry.createWeb3Card`
Status: Fixed in `bbaa845`

Description

Both functions checked `status == Approved` but did not advance status or set any guard. They could be called arbitrarily many times, emitting duplicate `StripeCardLinked` / `Web3CardCreated` events and misleading off-chain indexers or 0G proof aggregators that treat these events as unique lifecycle milestones.

Fix Applied

```
    struct CardRequest {
        ...
+       bool cardLinked;
    }

    function linkStripeCard(...) {
        ...
+       require(!cardRequest.cardLinked, "card already linked");
+       cardRequest.cardLinked = true;
```

F-9 · Low · `requestId` Slot Poisoning via Front-Running · OPEN

Location: `AgiCardsRegistry.requestCard`
Confidence: 75
Status: Open

Description

`requestCard` enforces uniqueness via `require(requests[requestId].createdAt == 0, "request exists")` on a caller-supplied `bytes32 requestId`. Any other agent owner who observes the pending transaction in the mempool can front-run it with the same `requestId` (using their own `agentId`), causing the victim's transaction to revert. Impact is limited to a transient DoS — the victim retries with a different identifier.

Recommended Fix

Derive `requestId` on-chain from the caller's address and a nonce to remove the squattable slot:

```
mapping(address => uint256) public nonces;

function requestCard(...) external onlyAgentOwner(agentId) {
    bytes32 requestId = keccak256(abi.encodePacked(msg.sender,
nonces[msg.sender]++));
    ...
}
```

---

6. Leads & Design Observations

Self-Approval Provides No Independent Security Guarantee

`requestCard` stores `owner: msg.sender` and `approveCardRequest` is gated by `requestOwner` (same address). The approval step is a lifecycle marker only — no independent party verifies the request. This is consistent with a single-user agent model but should be explicitly documented. If third-party authorization is required, an `approver` role must be introduced.

Timestamp Bucket for Daily Limit

`block.timestamp / 1 days` is manipulable by validators by up to ~12 seconds. Edge-case requests near UTC midnight may be credited to the wrong day bucket. This is negligible for any realistic daily limit but worth noting for high-frequency, high-value deployments.

---

7. Test Coverage

A Foundry test suite was added as part of the remediation (`test/AgiCardsRegistry.t.sol`).

Category	Count
Unit tests	28
Reentrancy attack simulation	1
Invariant checks	3
Fuzz tests	0

All 28 tests pass. Every fixed finding has at least one dedicated regression test.

Gaps remaining: no stateful fuzz testing of the `depositedBalance ≥ reservedBalance` invariant; no formal verification of the request state machine.

---

8. Disclaimer

> This report was produced by an AI-assisted audit pipeline. AI analysis cannot verify the complete absence of vulnerabilities. No guarantee of security is given. Independent human review, a bug bounty program, and on-chain monitoring are strongly recommended before and after deployment.
>
> For a consultation, visit https://www.pashov.com